ComicCraft – Preparing the Application for Local Development

Local Environment Setup, Running and Verification

Project: ComicCraft – AI Comic Story Creator
Technology: FastAPI, Jinja2, Gemini models, Stable Diffusion, FPDF
Topic covered: Preparing and testing ComicCraft on a local computer

This assignment is organized as a standalone technical note with clear concepts, implementation points, workflow diagrams, and practical checkpoints.

Prepared for Academic Submission

1. Preparing the Application for Local Development

Local development means running and testing ComicCraft on a developer's own computer before deploying it to a cloud or production environment. This stage is important because it allows developers to verify the complete application safely and repeatedly.

Basic Setup Flow

Install Python	Create venv	Install Dependencies	Configure Keys	Run FastAPI	Test

Required Development Tools

- Python and pip for application code and package management.
- A code editor such as VS Code or another suitable IDE.
- Git for version control and collaboration.
- FastAPI and Uvicorn for the web application and local server.
- Required AI and image-generation libraries.
- A browser for testing the user interface and FastAPI documentation.

Why Use a Virtual Environment?

A virtual environment isolates project dependencies from other Python projects. This reduces version conflicts and makes it easier to reproduce the same setup on another computer.

2. Virtual Environment and Dependency Installation

Create and Activate the Environment

```
python -m venv env # Windows env\Scripts\activate # macOS / Linux source env/bin/activate
```

Install Project Dependencies

The project document lists packages such as FastAPI, Uvicorn, Jinja2, python-multipart, Google Generative AI support, Diffusers, Transformers, FPDF, Pillow and Accelerate. A requirements.txt file is recommended so the environment can be reproduced consistently.

```
pip install -r requirements.txt
```

Recommended Dependency Practices

- Keep a requirements.txt file updated with the project's tested dependencies.
- Avoid installing unnecessary packages globally.
- Use a fresh virtual environment when troubleshooting dependency conflicts.
- Record important Python and package versions for reproducibility.

Project Structure

Folder / File	Purpose
app/main.py	Creates or exposes the FastAPI application.
routes.py	Contains route handlers and workflow orchestration.
templates/	Stores Jinja2 HTML templates.
static/panels/	Stores generated comic panel images.
static/exports/	Stores generated PDF files.
requirements.txt	Lists Python dependencies.
.env	Stores local environment variables; should not be committed when it contains secrets.

3. Configuring Environment Variables and Application Settings

AI applications often require API credentials. These values should be kept outside the source code whenever possible. Environment variables or a local .env file can be used to provide configuration during development.

```
Example Local Configuration
```

```
GEMINI_API_KEY=your_gemini_api_key_here
HF_API_KEY=your_huggingface_api_key_here
```

Important Security Practices

- Never hard-code real API keys directly into Python source files.
- Do not upload secrets to public Git repositories.
- Keep .env in .gitignore when it contains credentials.
- Use placeholder values in documentation and sample configuration files.
- Rotate a credential if it is accidentally exposed.

Jinja2 Frontend Integration

FastAPI uses Jinja2Templates to render HTML dynamically. The homepage collects user information, the preview page displays the generated comic, and the success page confirms PDF export.

```
from fastapi.templating import Jinja2Templates templates =
Jinja2Templates(directory="templates")
```

Configuration Checklist

- Python interpreter selected correctly in the IDE.
- Virtual environment activated.
- Dependencies installed successfully.
- Required environment variables available.
- Templates and static folders located in expected paths.

4. Running and Testing the Local Application

Start the FastAPI Server

From the project root, launch the development server with Uvicorn. The --reload option watches for code changes and restarts the development server automatically.

```
uvicorn app.main:app --reload
```

Access the Application

- Open the local application address: http://127.0.0.1:8000
- Use the homepage to enter a story prompt and comic preferences.
- FastAPI's interactive API documentation is available at /docs.

Suggested Testing Sequence

Open Home	Enter Prompt	Submit	Generate	Review Preview	Export PDF

Check	Expected Result
Server start	No startup exception; Uvicorn reports the application is running.
Homepage	index.html loads correctly.
Form submission	FastAPI receives the expected fields.
AI generation	Outline, story and images are produced.
Preview	Panels appear in correct order with text and images.
Export	PDF file is created in the expected export directory.

Local development documentation commonly recommends checking the project's README, dependency files and startup commands before running the application. ■cite■turn0search1■

5. Troubleshooting, Verification and Final Checklist

Before considering local deployment complete, the developer should verify both the environment and the application workflow. Problems are easier to solve when each layer is tested separately.

Common Problems and Solutions

Problem	Possible Cause	Action
Module not found	Dependency missing or wrong environment	Activate the virtual environment and reinstall requirements.
Server will not start	Import or configuration error	Read the terminal traceback and check app.main:app.
Template not found	Incorrect templates path	Verify the templates directory and Jinja2 configuration.
Image not generated	Model/API problem	Test /test-image separately and check credentials/configuration.
PDF export fails	Missing image or file permission issue	Check image paths and export directory.
Changes not visible	Browser cache or server state	Refresh the page and verify the reload server is active.

Final Verification Checklist

- ■ Virtual environment created and activated.
- ■ Dependencies installed from requirements.txt.
- ■ Environment variables configured safely.
- ■ FastAPI server starts with Uvicorn.
- ■ Homepage loads successfully.
- ■ /docs API documentation opens.
- ■ AI text generation works.
- ■ Image generation works.
- ■ Comic preview renders correctly.
- ■ PDF export completes successfully.

Conclusion

Preparing ComicCraft for local development creates a controlled environment in which the complete AI comic pipeline can be tested. A clean virtual environment, reproducible dependencies, safe configuration, clear project structure and systematic testing together form a reliable foundation for future deployment.

References: Supplied ComicCraft project document; GitHub's local-development guidance; Google Cloud's developer-environment guidance. ■cite■turn0search1■turn0search0■